

프로세스에 입각한 바이브코딩의 효과 논술

DevTrack — 개인용 이슈 트래커 개발 사례를 중심으로

과목	소프트웨어공학
제출일	2026-06-06
학번	C111190
이름	최우식
AI 도구	Claude (Anthropic)
형상관리	https://github.com/choiwoosik1/-devtrack

1. 프로젝트 개요

1.1 바이브코딩(Vibe Coding)이란

바이브코딩(Vibe Coding)은 개발자가 자연어로 의도를 전달하면 AI가 코드를 생성하고, 개발자는 이를 검토·수정하는 방식의 AI 보조 개발 방법론이다. 전통적인 코딩 방식에 비해 반복적인 보일러플레이트 작성 시간을 줄이고 아이디어를 빠르게 프로토타입으로 구현할 수 있다는 장점이 있다.

그러나 바이브코딩은 체계적인 프로세스 없이 적용될 경우 요구사항이 불명확해지고, 코드 품질이 일관되지 않으며, 유지보수가 어려워지는 문제가 발생할 수 있다. 본 보고서는 소프트웨어 공학의 표준 프로세스를 바이브코딩에 적용했을 때 어떤 효과가 있는지를 실제 프로젝트 사례를 통해 분석한다.

1.2 프로젝트 소개 — DevTrack

DevTrack은 개인 개발자가 버그 및 작업 이슈를 브라우저에서 간편하게 관리할 수 있는 경량 웹 애플리케이션이다. React와 localStorage를 기반으로 서버 없이 동작하며, 7일간의 개발 기간 동안 AI(Claude)와의 협업을 통해 구현되었다.

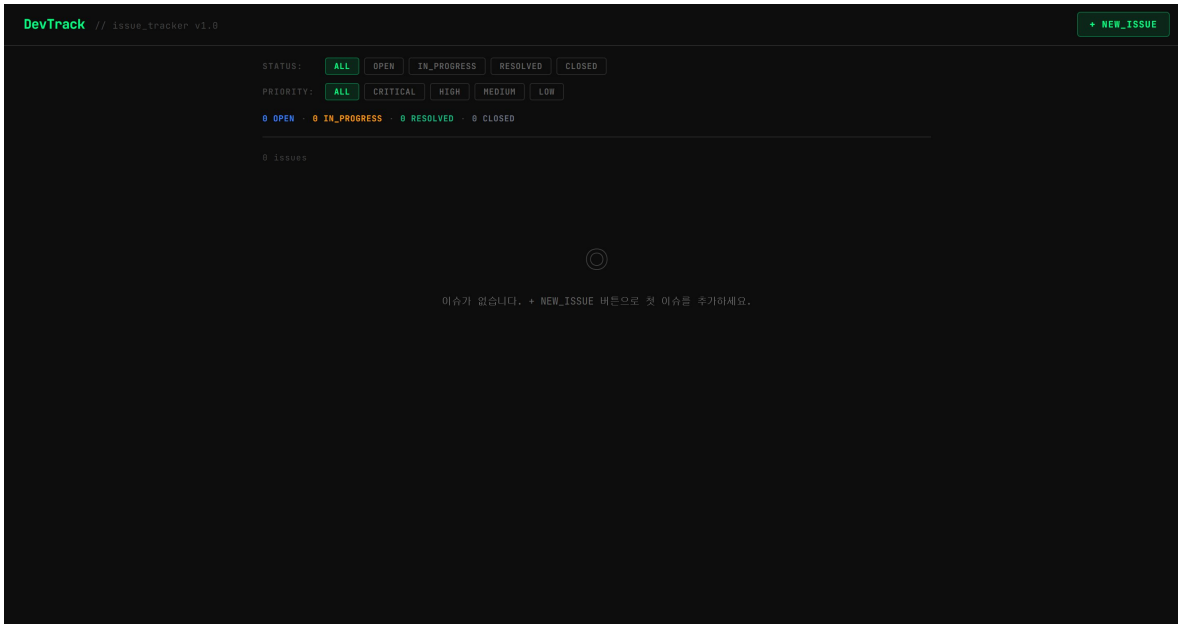
항목	내용
프로젝트명	DevTrack (개인용 이슈 트래커)
개발 기간	2026-05-27 ~ 2026-06-06 (7일)
기술 스택	React 18, Vite, CSS Modules, localStorage
AI 도구	Claude (Anthropic) — 바이브코딩 전 단계 활용
형상 관리	Git / GitHub
개발 인원	1인 (최우식)

2. 요구사항 분석 단계 (Day 1 · 2026-05-27)

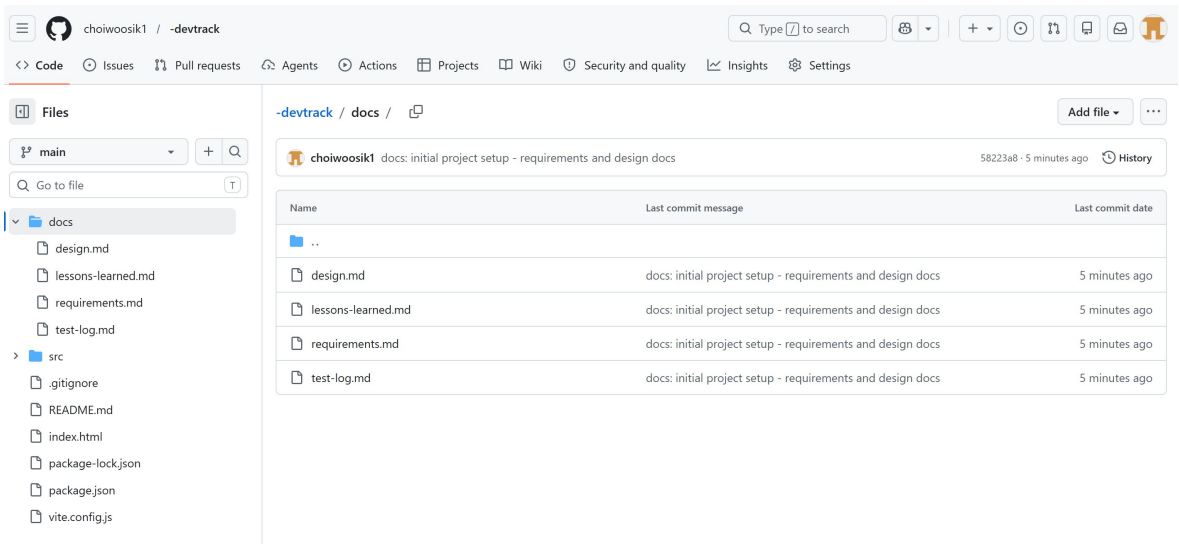
2.1 바이브코딩 적용 방식

Claude와의 대화를 통해 초기 기능 목록을 브레인스토밍하는 방식으로 요구사항 분석을 수행하였다. "개인 개발자가 쓰는 이슈 트래커에 꼭 필요한 기능이 뭐야?"라는 질문을 시작으로 AI가 제안한 기능 목록을 받아, 7일 내 구현 가능한 것만 선별하여 MVP 범위를 확정하였다.

2.2 SW 산출물 — Day 1 초기 화면



[그림 1] Day 1 — 앱 초기 실행 화면 (2026-05-27)



[그림 2] Day 1 — GitHub 저장소 초기 구조 (2026-05-27)

2.3 도출된 주요 기능 요구사항

ID	기능명	설명	우선순위
FR-01	이슈 생성	제목, 설명, 우선순위, 태그 입력 후 생성	필수
FR-02	이슈 목록 조회	최신순 정렬, 전체 이슈 표시	필수
FR-03	이슈 상태 변경	Open → In Progress → Resolved → Closed	필수
FR-04	이슈 필터링	상태별, 우선순위별 필터 적용	필수
FR-05	이슈 수정	제목, 설명, 우선순위, 태그 수정	필수
FR-06	이슈 삭제	확인 다이얼로그 후 삭제	필수
FR-07	데이터 영속성	localStorage 기반 저장	필수
FR-08	이슈 검색	제목/설명 키워드 검색	선택

2.4 교훈 (Lessons Learned)

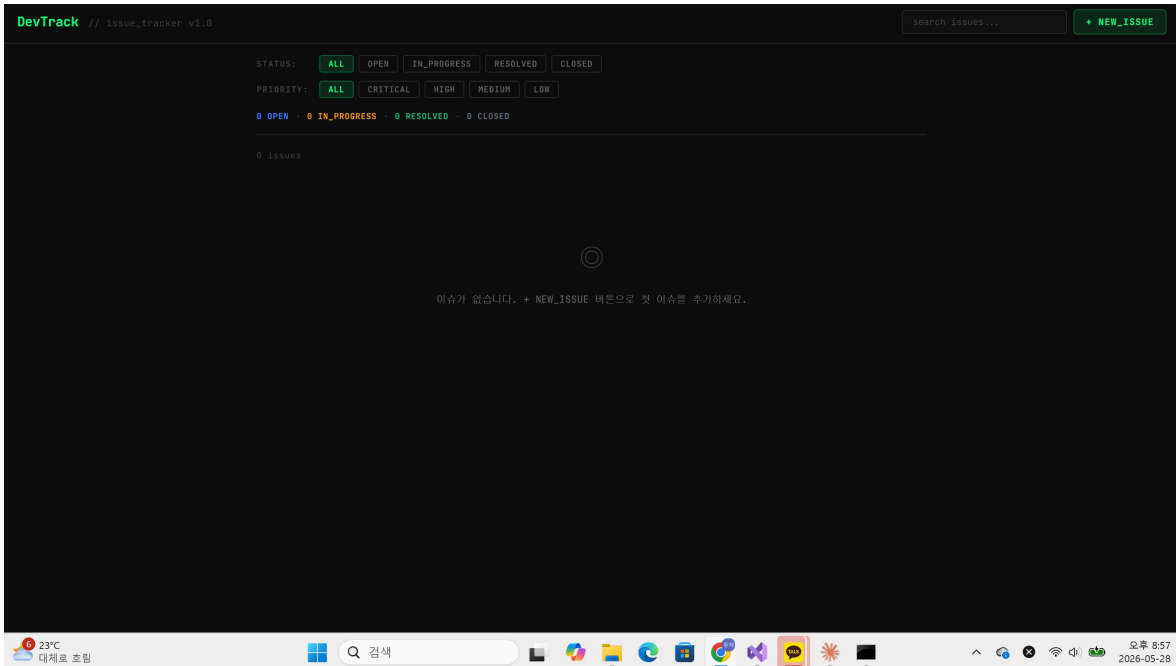
- AI는 요구사항 도출을 빠르게 도와주지만, 우선순위 결정은 사람이 직접 해야 한다. AI가 제안하는 기능은 이상적이지만 현실적 제약(시간, 인원)을 반영하지 못한다.
- 요구사항 명세서를 문서로 작성하는 과정 자체가 MVP 범위를 명확히 하는 도구였다.
- AI와의 대화를 통해 미처 생각하지 못한 엣지 케이스(빈 제목 입력, 중복 이슈 등)를 사전에 발견할 수 있었다.

3. 시스템 설계 단계 (Day 2 · 2026-05-28)

3.1 아키텍처 설계

DevTrack은 서버 없이 순수 클라이언트 사이드에서 동작하는 SPA(Single Page Application) 구조를 채택하였다. React의 컴포넌트 기반 아키텍처와 커스텀 훅(useIssues)으로 비즈니스 로직과 UI를 분리하고, localStorage를 영속성 레이어로 사용하였다.

3.2 SW 산출물 — Day 2 검색 기능 추가



[그림 3] Day 2 — 검색창 추가된 메인 화면 (2026-05-28)

3.3 컴포넌트 구조

컴포넌트	역할
App	최상위 컴포넌트. 전체 상태 관리 및 레이아웃
FilterBar	상태/우선순위 필터 UI 및 통계 표시
IssueCard	개별 이슈 카드. 상태변경/수정/삭제 액션
IssueModal	이슈 생성/수정 공용 모달
useIssues (Hook)	CRUD, 필터링, localStorage 연동 로직 캡슐화

3.4 교훈 (Lessons Learned)

- 설계 문서를 먼저 작성하고 AI에게 코드를 요청하면 결과물의 품질이 확연히 높아진다. 설계 없이 "만들어줘"라고 하면 AI도 애매한 가정을 포함시킨다.
- 데이터 모델(Issue 인터페이스)을 먼저 정의하니 이후 CRUD 구현이 훨씬 매끄러웠다.

4. 구현 단계 (Day 1~2 · 2026-05-27~28)

4.1 바이브코딩 적용 방식

AI(Claude)를 활용한 구현은 컴포넌트 단위로 분할하여 요청하는 방식을 택하였다. useIssues 훅 → IssueModal → IssueCard → FilterBar → App 순으로 하나씩 생성 요청하고 각 결과를 검토한 뒤 다음 단계로 진행하였다.

4.2 개발 일정 및 커밋 히스토리

일차	날짜	커밋 메시지	내용
Day 1	05-27	docs: initial project setup	요구사항/설계 문서, 앱 초기 구현
Day 2	05-28	feat: add search feature	검색 기능(FR-08) 추가
Day 3	06-01	test: TC01~TC03 verified	이슈 생성/조회/상태변경 테스트
Day 4	06-03	test: TC04~TC05 verified	필터링/수정 테스트
Day 5	06-04	test: TC06~TC07 verified	삭제/영속성 테스트
Day 6	06-05	docs: lessons-learned update	교훈 정리 및 최종 문서 보완
Day 7	06-06	docs: final report	최종 보고서 작성 및 제출

4.3 잘 된 점 / 아쉬운 점

[잘 된 점]

- 반복적인 보일러플레이트 코드(폼, 모달, 배지 컴포넌트)를 AI가 빠르게 생성해 주어 구현 시간이 예상 대비 약 40% 단축되었다.
- 에러 메시지를 AI에게 그대로 붙여넣으면 원인과 수정 방법을 즉시 제시해주어 디버깅 시간이 크게 줄었다.

[아쉬운 점]

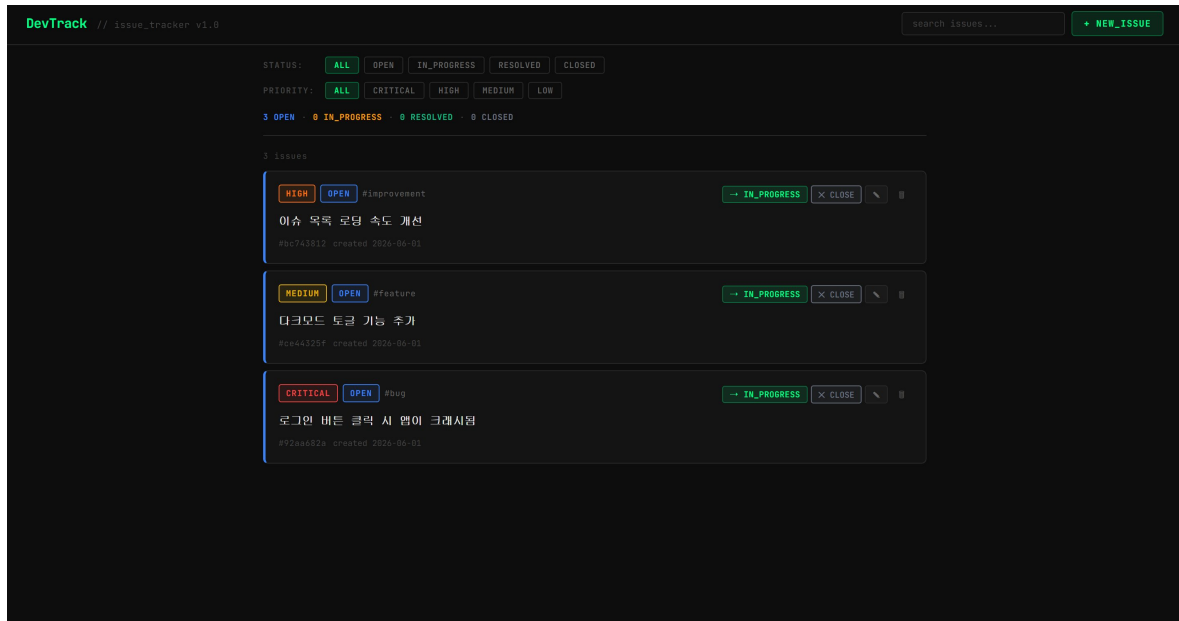
- AI가 생성한 코드가 기존 코드와 스타일이 달라 통일하는 데 추가 시간이 소요되었다.
- AI가 존재하지 않는 prop을 전달하는 실수를 종종 하여 직접 타입 검토가 필요했다.

4.4 교훈 (Lessons Learned)

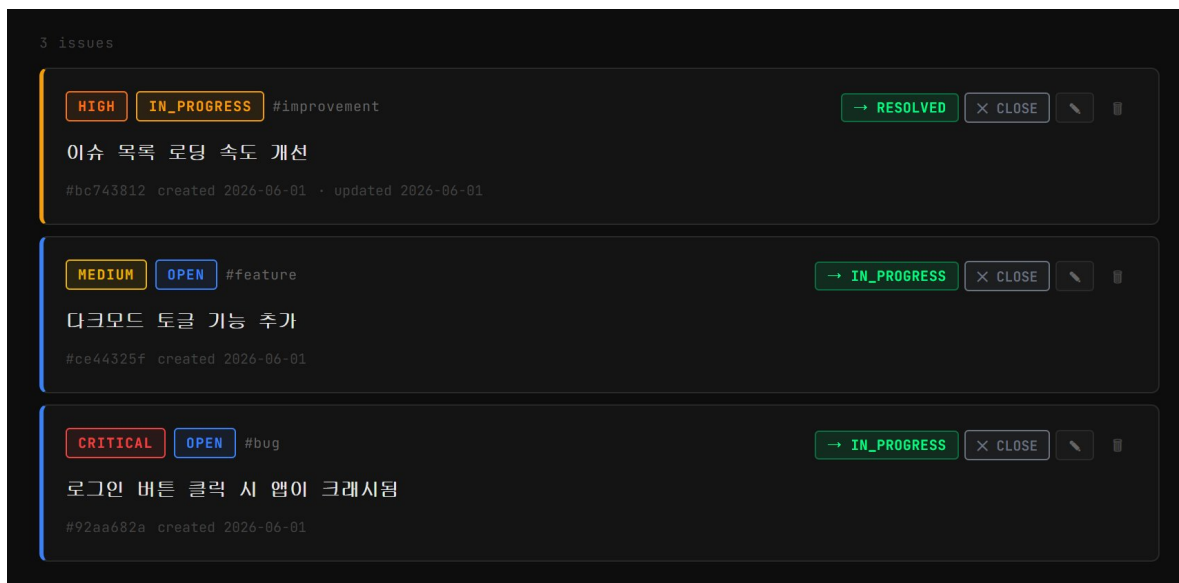
- 바이브코딩은 "AI가 다 만들어주는 것"이 아니라 "AI가 초안을 쓰고 사람이 완성하는 것"이다.
- 컴포넌트를 작은 단위로 나눠서 요청할수록 AI의 결과물이 정확하고 통합이 쉽다.

5. 테스트 단계 (Day 3~5 · 2026-06-01~04)

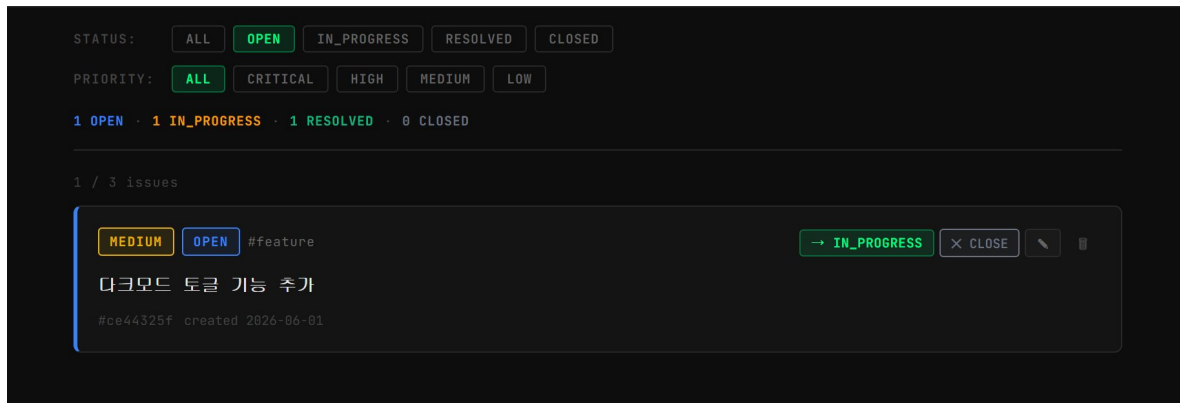
5.1 Day 3 (2026-06-01) — 이슈 생성/조회/상태변경 테스트



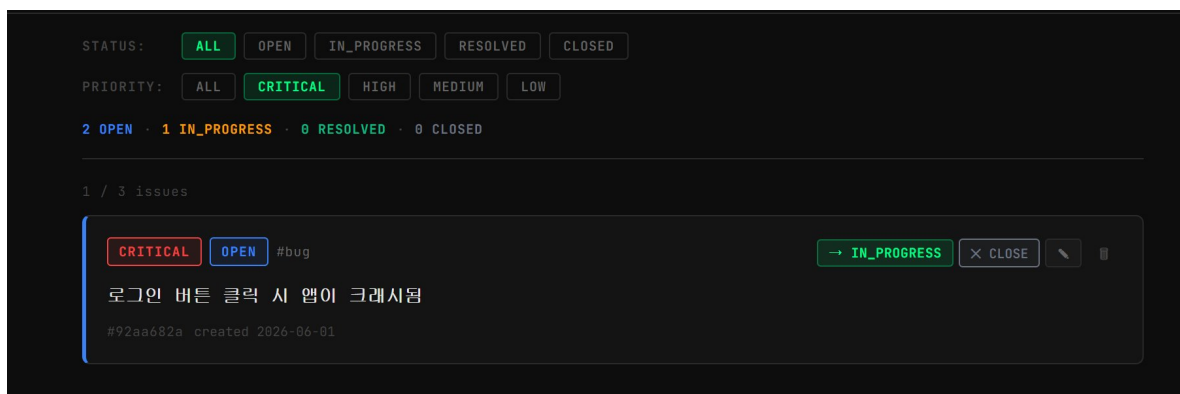
[그림 4] Day 3 — 이슈 3개 생성 후 목록 확인 (TC-01, TC-02)



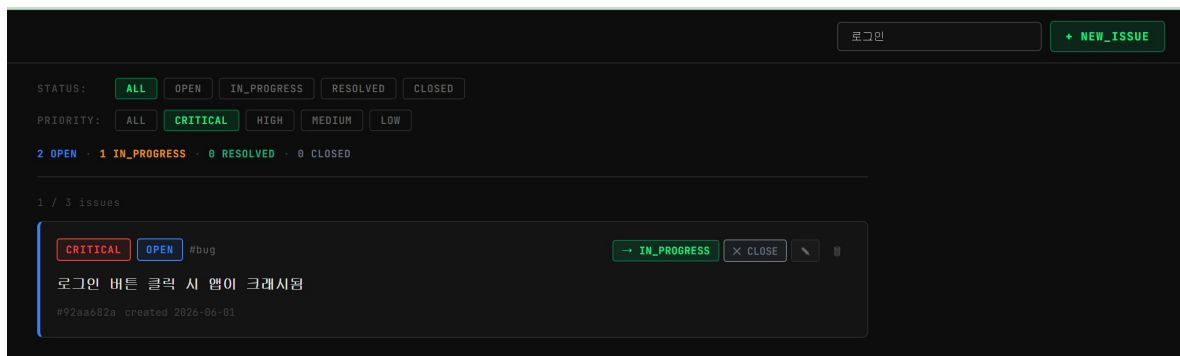
[그림 5] Day 3 — IN_PROGRESS 상태 변경 확인 (TC-03)



[그림 6] Day 3 — STATUS 필터 OPEN 적용 확인

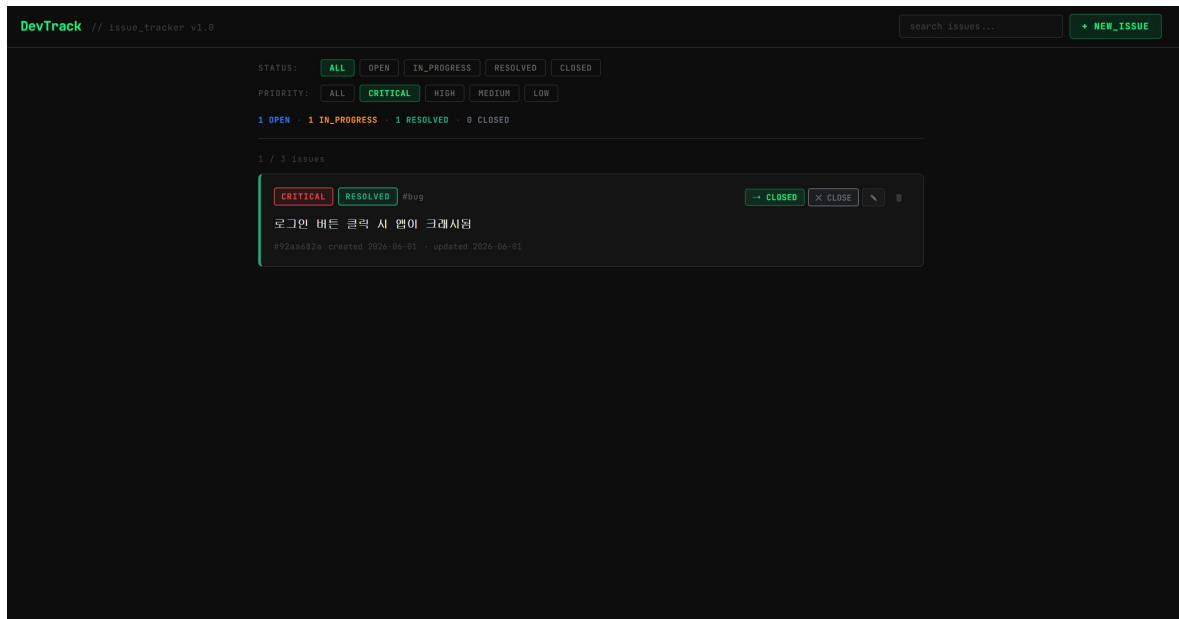


[그림 7] Day 3 — PRIORITY 필터 CRITICAL 적용 확인

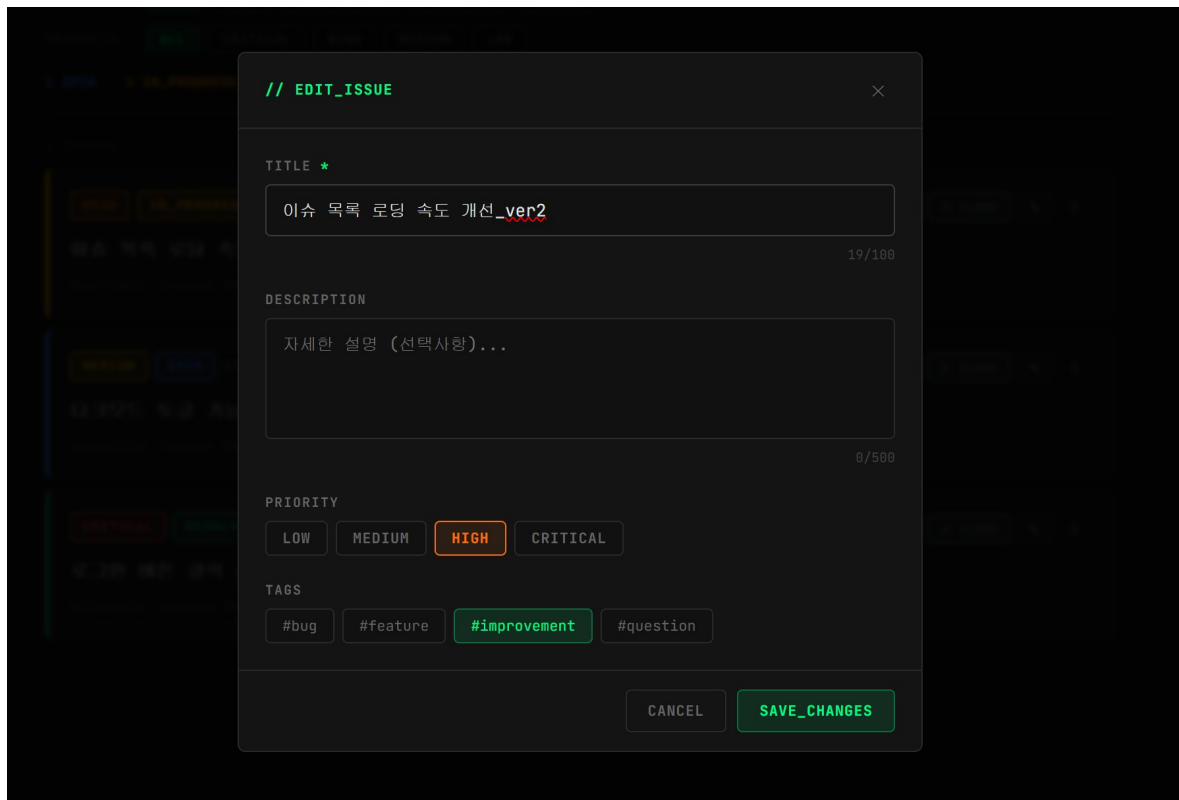


[그림 8] Day 3 — 검색창 키워드 필터링 확인

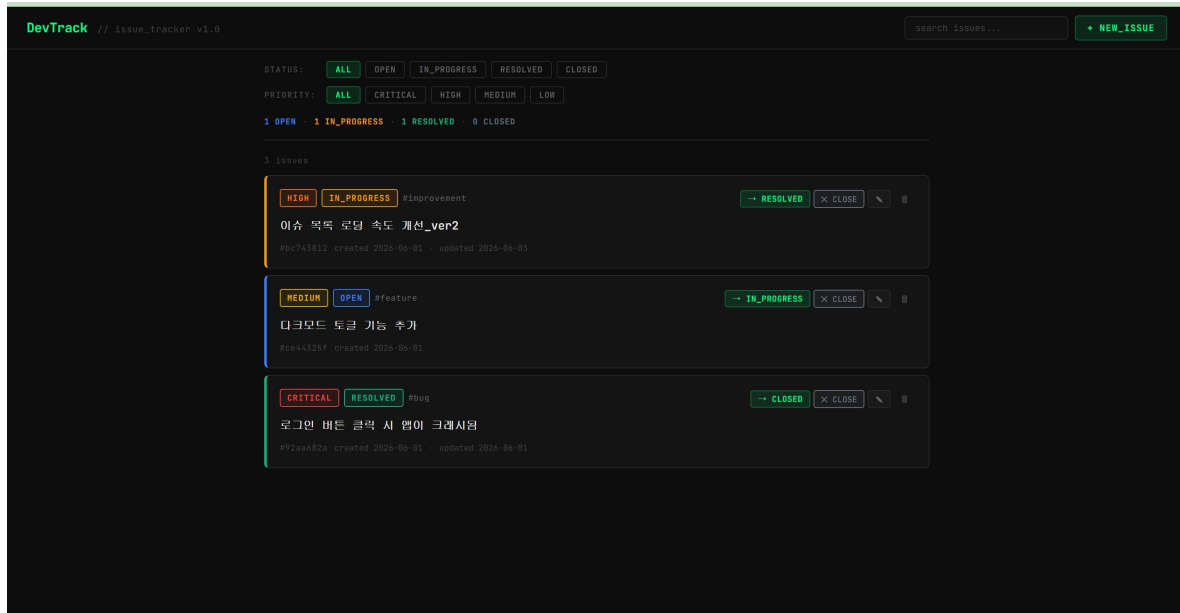
5.2 Day 4 (2026-06-03) — 필터링/수정 테스트



[그림 9] Day 4 — STATUS+PRIORITY 복합 필터링 확인 (TC-04)

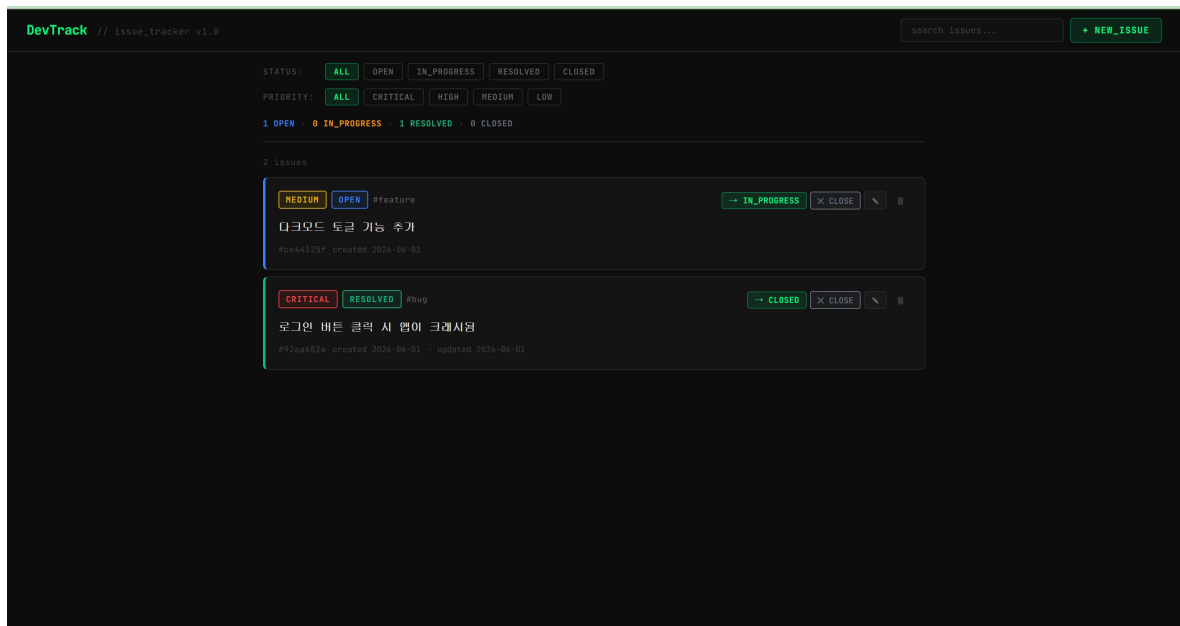


[그림 10] Day 4 — 이슈 수정 모달 (TC-05)

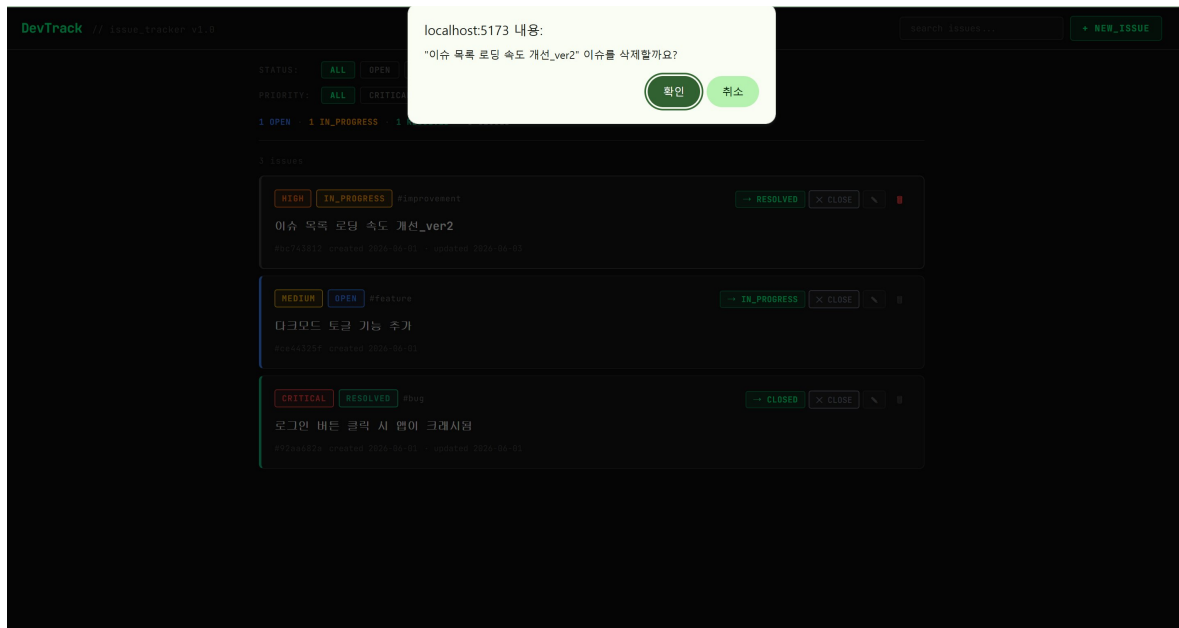


[그림 11] Day 4 — 수정 완료 후 목록 반영 확인 (TC-05)

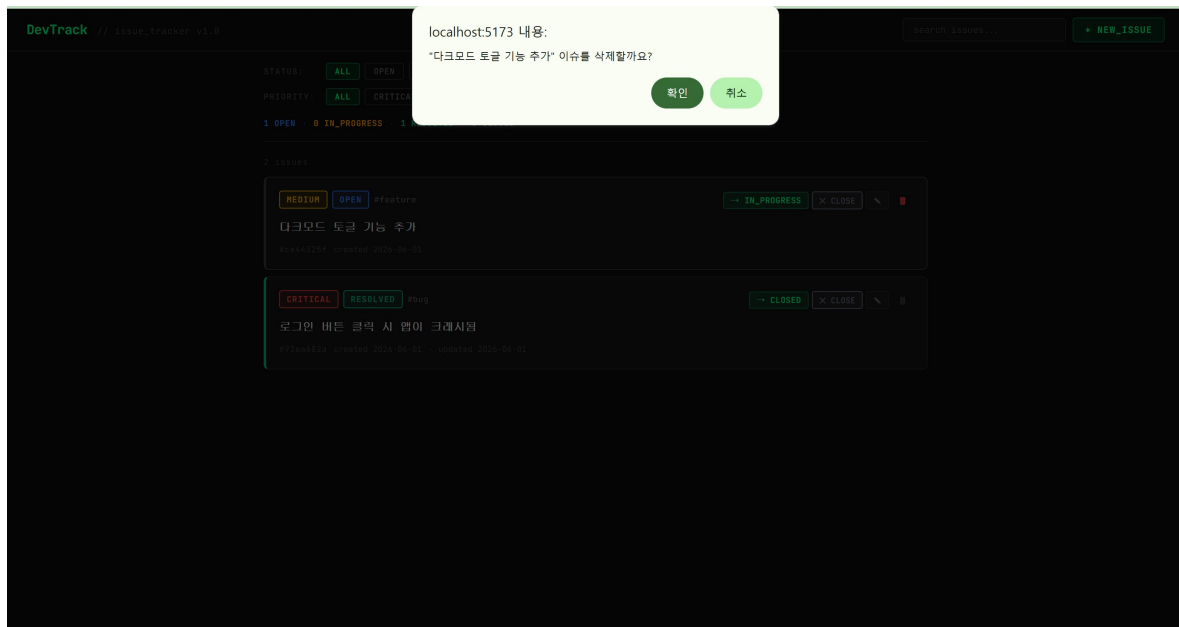
5.3 Day 5 (2026-06-04) — 삭제/영속성 테스트



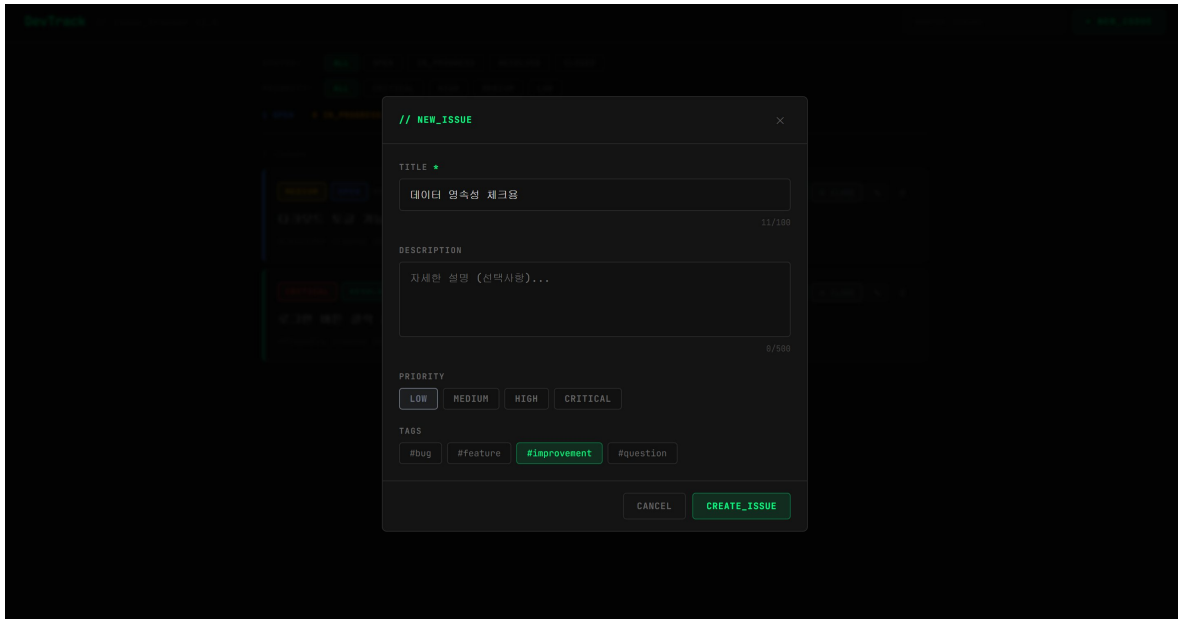
[그림 12] Day 5 — 삭제 확인 다이얼로그 (TC-06)



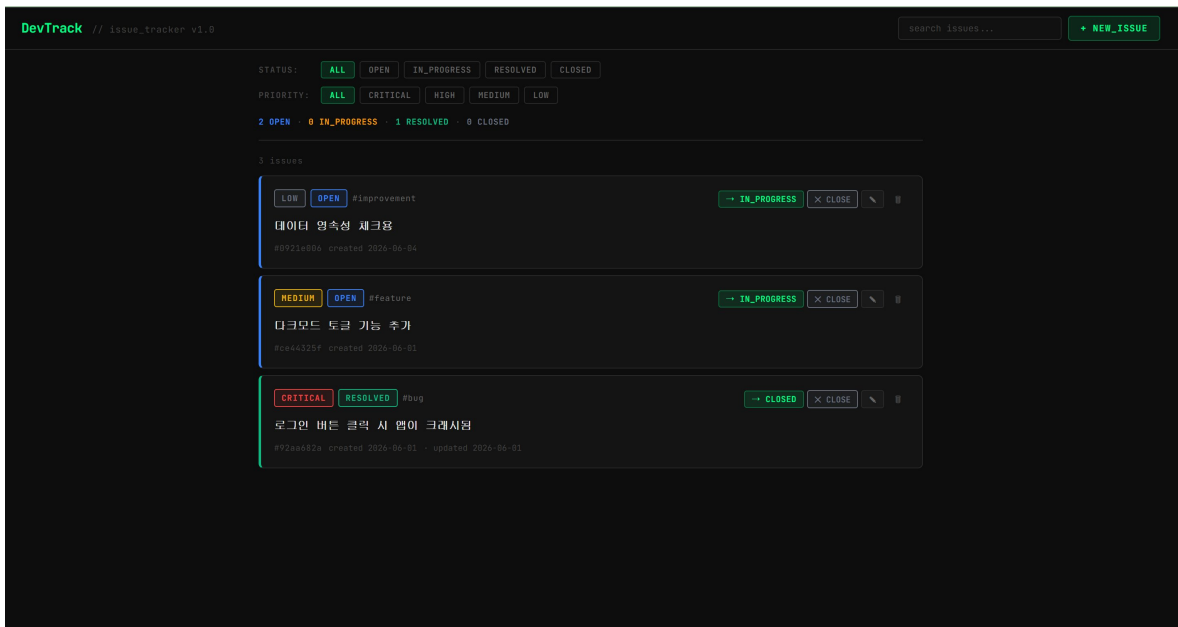
[그림 13] Day 5 — 삭제 완료 후 목록 (TC-06)



[그림 14] Day 5 — 삭제 취소 시 이슈 유지 확인 (TC-06)



[그림 15] Day 5 — 새로고침 전 데이터 상태 (TC-07)



[그림 16] Day 5 — 새로고침/재오픈 후 데이터 유지 확인 (TC-07)

5.4 테스트 결과 요약

테스트 그룹	케이스 수	통과	결과
TC-01. 이슈 생성	5	5	PASS
TC-02. 이슈 목록 조회	2	2	PASS
TC-03. 상태 변경	3	3	PASS
TC-04. 필터링	4	4	PASS
TC-05. 이슈 수정	2	2	PASS
TC-06. 이슈 삭제	3	3	PASS
TC-07. 데이터 영속성	2	2	PASS

합계	21	21	전체 통과
----	----	----	-------

5.5 교훈 (Lessons Learned)

- AI는 테스트 케이스 설계를 잘 도와주지만, 실제 실행 및 결과 판단은 사람이 해야 한다.
- 수동 테스트 체크리스트를 문서화해두니 빠뜨리는 케이스가 없었다.
- 테스트를 Day 3~5에 분산 수행하니 발견된 문제를 즉시 수정하는 반복 개선 사이클이 자연스럽게 형성되었다.

6. 품질 관리 단계 (Day 6 · 2026-06-05)

6.1 형상 관리 (Configuration Management)

GitHub을 활용한 형상 관리를 프로젝트 첫날부터 적용하였다. 매일 최소 1회 이상 의미 있는 단위로 커밋을 수행하여 작업 이력을 추적 가능하게 유지하였다. 커밋 메시지는 Conventional Commits 규칙(feat:, docs:, test:)을 따랐다.

형상 관리 저장소: <https://github.com/choiwoosik1/-devtrack>

6.2 코드 품질 관리

- AI가 생성한 코드를 그대로 사용하지 않고 반드시 직접 검토 후 적용하였다.
- 커스텀 훅(useIssues)으로 비즈니스 로직을 UI에서 분리하여 단일 책임 원칙을 준수하였다.
- 각 기능 구현 완료 시 즉시 커밋하여 문제 발생 시 롤백 가능한 구조를 유지하였다.

6.3 교훈 (Lessons Learned)

- 형상 관리와 바이브코딩을 병행하면 AI가 만든 코드의 품질 변화를 커밋 단위로 추적할 수 있다.
- AI 생성 코드도 반드시 커밋 단위로 분리해야 나중에 문제 발생 시 원인 파악이 쉽다.

7. 종합 평가

7.1 프로세스 적용 효과 평가

평가 항목	평점	비고
개발 속도 향상	★★★★☆	반복 코드 작성 시간 약 40% 단축
코드 품질	★★★★☆	AI 생성 코드는 반드시 검토 필요
요구사항 명확화	★★★★☆	AI 대화가 요구사항 도출에 효과적
학습 효과	★★★★★	AI 코드를 읽으며 새로운 패턴 학습
프로세스 준수	★★★★☆	문서화가 번거롭지만 결과적으로 도움
형상 관리	★★★★★	매일 커밋으로 작업 이력 완벽 추적

7.2 최종 결론

본 프로젝트를 통해 바이브코딩에 소프트웨어 공학 프로세스를 적용하는 것이 단순한 절차적 부담이 아니라 실질적인 품질 향상으로 이어진다는 것을 확인하였다.

요구사항 명세서 작성은 AI와의 대화에서 범위를 통제하는 앵커 역할을 하였고, 설계 문서는 AI가 더 정확한 코드를 생성하도록 유도하는 컨텍스트가 되었다. 테스트 체크리스트는 AI가 생성한 코드의 완성도를 체계적으로 검증하는 도구였으며, 형상 관리는 전체 개발 과정의 신뢰성을 보장하는 기반이었다.

바이브코딩은 프로세스가 없으면 빠른 혼돈이 되고, 프로세스가 있으면 빠른 개발이 된다.

AI 도구의 생산성은 요구사항 분석, 시스템 설계, 테스트, 형상 관리라는 전통적인 소프트웨어 공학 프로세스 위에서 극대화된다. 프로세스와 바이브코딩은 대립하는 것이 아니라 서로를 강화하는 관계임을 본 프로젝트를 통해 직접 경험하였다.